WEB DEVELOPMENT · 3RD SEMESTER

Week 02 · Block 2a

Writing HTML

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Explain what **HTML** is — and what it is *not*
- Read and write the parts of an **element**: tags, attributes, content
- Lay out a valid **document skeleton** from memory
- Use the **core elements**: headings, paragraphs, links, images, lists
- Choose **semantic** tags over `<div>` — and say why that helps **accessibility**

Remember the body of that `200 OK` response?

That was HTML.

Last block we watched it arrive. Now we write it ourselves.

PART 1

What is HTML?

HyperText Markup Language

Markup, not programming

HTML **describes the structure and meaning** of content. It is not a programming language — there is no logic, no loops, no variables.

- You **mark up** text to say *what each part is*: "this is a heading", "this is a link"
- The browser reads that markup and builds the page
- **Structure and meaning** — *not* appearance. (Looks come from CSS, in Block 3.)

This is closer to writing a data structure than writing code. You describe *what things are*; nothing "runs".

HyperText: the ”web” in web

Ordinary **text** is linear — you read it start to finish, like a book.

HyperText adds one radical idea: **links**. Any word can point to another document, anywhere on the internet.

```
<a href="https://developer.mozilla.org">read more over here</a>
```

- Each link leads to another document, which links onward — weaving isolated pages into a **web** (that's where the name comes from)
- The `<a>` (**anchor**) element *is* the link: the most important element on the web

This "jump anywhere" idea (Tim Berners-Lee, 1989) is what makes the web a *web*, not a pile of separate documents.

Anatomy of an element

```
<a href="https://hs-ansbach.de">Our university</a>
```

- `<a …>` — the **opening tag**; `a` is the **element name** (here: a link)
- `href="…"` — an **attribute**: a `name="value"` pair that configures the element
- `Our university` — the **content**: what the user sees
- `</a>` — the **closing tag**: same name, with a slash

Elements almost always come in **open / close** pairs that wrap their content.

Void elements

A few elements have **no content**, so they have **no closing tag**:

```
<img src="logo.svg" alt="Course logo" />
<br />
<meta charset="UTF-8" />
```

- `<img>` — an image. Everything it needs is in its attributes.
- `<br>` — a line break.
- `<meta>` — a piece of metadata.

These are called **void** (or empty) elements. The trailing `/` is optional but tidy.

PART 2

The Document Skeleton

Every page starts the same way

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, web!</h1>
    <p>This is my page.</p>
  </body>
</html>
```

Learn this by heart — you'll type it at the top of every file you write.

Head vs. body

`<head>` — *about* the page

- Not shown on the page itself
- `<title>` — text in the browser tab
- `<meta charset>` — how to read the bytes
- Later: links to CSS and JS

`<body>` — *the* page

- Everything the user actually sees
- Headings, text, links, images, …
- This is where you'll spend your time

`<!DOCTYPE html>` just says "this is modern HTML". `lang="en"` tells browsers and screen readers which language the page is in.

PART 3

The Building Blocks

Headings & paragraphs

```
<h1>Week 2</h1>
<p>Previously, you learnd how the web works. Now we ...</p>

<h2>Block 2a – Basic HTML</h2>
<p>In this block you meet the first basic elements of ...</p>
```

- `<h1>` – `<h6>` — six levels of heading, forming an **outline**
- **One `<h1>` per page** (the page's title); don't skip levels for looks
- `<p>` — a paragraph of text

Headings are an outline, not font sizes. Screen-reader users jump *heading to heading* — so the hierarchy has to make sense.

Links & images

```
<a href="week02.html">Go to Week 2</a>

<img src="logo.svg" alt="Web Development logo" />
```

- `<a href="…">` — a **link** (an *anchor*); `href` is where it points
- `<img src="…" alt="…">` — an **image**; `src` is the file
- `alt` describes the image for screen readers and for when it fails to load

`alt` is not optional. A missing `alt` is invisible to a sighted user but a dead end for someone using a screen reader.

Lists

Unordered — order doesn't matter

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Ordered — sequence matters

```
<ol>
  <li>Request</li>
  <li>Response</li>
  <li>Render</li>
</ol>
```

Every item is an `<li>` (list item), nested inside the list.

Elements nest — and form a tree

```html
<body>
  <main>
    <h1>Title</h1>
    <p>Some
       <a href="#">link</a>.
    </p>
  </main>
</body>
```

```
body
└─ main
   ├─ h1 "Title"
   └─ p
      ├─ "Some "
      └─ a "link"
```

An element inside another is its **child**. This tree is the page's structure — and in **Block 6** we'll manipulate exactly this tree with JavaScript (the DOM).

PART 4

Semantic HTML

Not everything is a `<div>`

`<div>` and `<span>` are **generic** boxes — they carry **no meaning**.

You *could* build a whole page from them… but nobody (and no machine) would know what any part *is*.

- `<div>` — a generic **block** (takes its own line)
- `<span>` — a generic **inline** piece (sits within text)

Reach for them only when **no meaningful element fits**.

Semantic elements = page landmarks

Semantic tags name the **role** each region plays:

```
┌──────────────────────────────────────────────┐
│  <header>     site title, logo               │
│  <nav>        the main menu                  │
├──────────────────────────────────────────────┤
│  <main>                                      │
│    <section>   a themed group of content     │
│    <article>   a self-contained piece        │
│    <aside>     related side content          │
├──────────────────────────────────────────────┤
│  <footer>     copyright, contact             │
└──────────────────────────────────────────────┘
```

Same page, two ways

`<div>` soup — meaningless

```
<div class="header">
  <div class="nav">…</div>
</div>
<div class="content">
  <div class="post">…</div>
</div>
<div class="footer">…</div>
```

Semantic — self-describing

```
<header>
  <nav>…</nav>
</header>
<main>
  <article>…</article>
</main>
<footer>…</footer>
```

Same look in the browser. Only the right side tells you — and the machine — *what each part is*.

Why it matters: accessibility

The meaning you write is what **assistive technology** relies on.

- **Screen-reader** users jump between **landmarks** (`main`, `nav`, …) and **headings** — `<div>` soup gives them nothing to jump to
- Search engines and browser "reader modes" use the same structure
- It costs you **nothing extra** — you were going to write a tag anyway; write the *meaningful* one

> This is the whole reason we lead with semantics: good structure *is* accessibility. It's a habit, not a feature you bolt on later.

PART 5

Write It Well

The browser is forgiving — you shouldn’t be

Browsers try hard to render even broken HTML. That's a trap: it may *look* fine and still be wrong.

- **Close your tags**, and nest them properly (`<p><strong>…</strong></p>`, not crossed)
- **Indent** to mirror the tree — future-you will thank you
- Check your work with the **W3C validator** (**<validator.w3.org>**)

Valid, well-structured HTML is the foundation everything else — CSS, JavaScript, accessibility — builds on.

Recap

- HTML is **markup**: it describes **structure and meaning**, not appearance
- An **element** = opening tag + attributes + content + closing tag *(some are void)*
- Every page shares a **skeleton**: `doctype`, `html`, `head`, `body`
- Core blocks: **headings, paragraphs, links, images, lists** — nesting into a **tree**
- Prefer **semantic** tags over `<div>` — because **structure is accessibility**

<EXERCISE> · 45 MIN

Your turn 👉

Hand-write a **multi-section, semantic** page about yourself — no CSS, just structure.

1. Start from the document **skeleton** (from memory!)
2. Use `<header>`, `<nav>`, `<main>`, `<footer>` as **landmarks**
3. Inside `<main>`: headings, paragraphs, a **list**, a **link**, and an **image** (with `alt`)
4. Validate it — zero errors at **validator.w3.org**

At home — Task 6: add a **second page** and link both pages to each other.

Starter: `exercises/week02/2a/` — open `README.md`. The folder has an image ready to use; the HTML is yours to write by hand.